

GitHub Fundamentals: A Complete Instructional Guide for Students

Table of Contents

1. Introduction
2. Chapter 1: Introduction to Git and GitHub
3. Chapter 2: Setting Up Git and GitHub
4. Chapter 3: Your First GitHub Repository
5. Chapter 4: The Essential Git Commands
6. Chapter 5: Branching and Merging
7. Chapter 6: Collaborating with Teams
8. Chapter 7: Managing Issues and Projects
9. Chapter 8: GitHub Best Practices
10. Chapter 9: GitHub Actions & Automation
11. Chapter 10: GitHub Pages and Hosting
12. Chapter 11: Advanced GitHub Features
13. Chapter 12: Contributing to Open Source Projects
14. Chapter 13: Troubleshooting and Common Issues
15. Conclusion
16. Appendix: Glossary, Resources, Cheat Sheet
17. Teaching Strategy

Introduction

Why Learn GitHub?

This section sets the stage for why every student or developer needs to learn GitHub, explaining its role in modern software development, the benefits of version control, and collaboration.

Chapter 1: Introduction to Git and GitHub

What is Git?

What is GitHub?

How GitHub is Different from Other Platforms

Basic Git Terminology: Repository, Commit, Branch, Clone, etc.

Exercises

1) Define a repository and explain its purpose.

Answer: A repository stores project files and history; it tracks changes and enables collaboration.

2) What is a commit?

Answer: A commit records a snapshot of changes with a message and metadata.

Chapter 2: Setting Up Git and GitHub

Installing Git on Your Computer

Configuring Git (Username, Email)

Setting Up a GitHub Account

Introduction to GitHub Desktop & GitHub CLI (optional)

Connecting Git to GitHub for the First Time

Exercises

1) Configure your Git username and email. Provide the commands.

Answer: `git config --global user.name "Your Name"; git config --global user.email "you@example.com"`

Chapter 3: Your First GitHub Repository

What is a Repository?

Creating a Repository on GitHub

Cloning a Repository to Your Local Machine

Initializing a Repository Locally

Making Your First Commit: A Hands-On Example

Exercises

1) Create, initialize, add, and commit a README.md locally. Show commands.

Answer: `echo "# Project" > README.md; git init; git add README.md; git commit -m "Add README"`

Chapter 4: The Essential Git Commands

- `git init`: Starting a New Project
- `git add`: Staging Files for Commit
- `git commit`: Recording Changes
- `git status`: Checking the State of Your Repository
- `git log`: Viewing Commit History
- `git push`: Uploading Changes to GitHub
- `git pull`: Getting Updates from GitHub

Exercises

1) What sequence of commands uploads local commits to GitHub?

Answer: `git add`, `git commit`, `git push`

Chapter 5: Branching and Merging

What is a Branch and Why Use It?

Creating and Switching Branches (git branch, git checkout)

Merging Branches: Bringing Changes Together

Resolving Merge Conflicts

Best Practices for Using Branches in Collaboration

Exercises

1) Create and switch to a new branch named feature-x.

Answer: git checkout -b feature-x

Chapter 6: Collaborating with Teams

How GitHub Makes Collaboration Easier

Forking Repositories

Creating Pull Requests (PRs)

Reviewing and Commenting on Pull Requests

Approving or Rejecting Changes

Merging PRs into Main Branch

Exercises

1) Describe the fork-and-PR workflow in steps.

Answer: Fork repo -> Clone -> Create branch -> Commit -> Push -> Open PR -> Address review -> Merge

Chapter 7: Managing Issues and Projects

What are GitHub Issues?

How to Create, Label, and Assign Issues

Managing Projects with GitHub Projects

Kanban Boards for Team Collaboration

Exercises

1) Create an issue and assign it to a team member. (Describe steps)

Answer: Click Issues -> New issue -> Fill title/body -> Assign -> Label -> Submit

Chapter 8: GitHub Best Practices

Writing Descriptive Commit Messages

Structuring Your Project for Clarity

Keeping Your Repository Clean and Organized

Using .gitignore Files to Manage Untracked Files

Keeping Commit History Clean (Squashing Commits, Rebase)

Exercises

1) When would you use rebase vs merge?

Answer: Rebase for linear history; merge to preserve branch history and context.

Chapter 9: GitHub Actions & Automation

Introduction to GitHub Actions

Setting Up Continuous Integration (CI) for Your Projects

Automating Tasks: Running Tests, Deploying Code

Example: Automating a Simple Workflow

Exercises

1) What is a simple CI workflow step to run tests on push? (Describe YAML)

Answer: name: CI; on: [push]; jobs: build: runs-on: ubuntu-latest; steps: - uses: actions/checkout@v2 - name: Run tests; run: pytest

Chapter 10: GitHub Pages and Hosting

What is GitHub Pages?

Creating a Personal or Project Website

Step-by-Step Guide to Publishing a Static Website on GitHub Pages

Customizing Your GitHub Pages Domain

Exercises

1) How do you publish a static site from the docs/ folder?

Answer: In repo settings -> Pages -> Source -> Deploy from a branch -> select branch and /docs folder -> Save

Chapter 11: Advanced GitHub Features

GitHub Pages for Blog & Documentation

GitHub Packages (for publishing your code libraries)

GitHub Insights (for analyzing your repositories)

GitHub API (for advanced automation and interaction)

Exercises

1) Give an example use-case for the GitHub API.

Answer: Automating issue creation or retrieving repo statistics for dashboards.

Chapter 12: Contributing to Open Source Projects

How to Find Open Source Projects

Forking, Cloning, and Contributing Code

Opening an Issue or Pull Request for Contribution

The GitHub Community: How to Communicate Effectively

Exercises

1) What should you include in a good PR description?

Answer: Summary, motivation, changes, testing instructions, related issues

Chapter 13: Troubleshooting and Common Issues

Dealing with Merge Conflicts

Fixing Broken Pushes (Force Push)

Recovering Lost Changes

What to Do When You Can't Push to GitHub

Exercises

1) How do you recover a lost commit?

Answer: Use `git reflog` to find commit SHA and `git checkout -b recovered <sha>` to restore

Appendix

Glossary of Terms

Additional Resources (Books, Websites, Tutorials)

Further Reading on GitHub Features

Cheat Sheet of Common Git Commands

Teaching Strategy

Hands-On Approach: Each chapter will come with practical exercises.

Project-Based Learning: Students will have the opportunity to work on collaborative projects and apply what they've learned.

Real-Life Scenarios: Encourage students to create personal or group projects on GitHub.

Peer Review & Collaboration: Foster a collaborative environment by integrating peer reviews and GitHub PRs.